

UnifAI / Raksha Enterprise AI Gateway & Guard

Technical Architecture & Specification Document

Document Version: 2.4.0

Target Systems: UnifAI Core, Raksha HTTP Gateway, Browser Guard Fleet, Management UI

Classification: Enterprise Technical Documentation

Target Audience: Software Architects, Lead Developers, Security Engineers, DevOps Engineers

1. Executive Summary & High-Level System Overview

UnifAI (incorporating the Raksha AI Security Suite) is an enterprise-grade AI Gateway, Governance Platform, and Endpoint AI Guardrail system. It addresses two critical enterprise AI challenges in a unified architecture:

- 1. Centralized AI Gateway & Governance (Server Plane):** Provides a unified, high-performance API gateway that unifies over 28 commercial, open-source, and private AI providers (OpenAI, Anthropic, Google Gemini, AWS Bedrock, Ollama, Cohere, DeepSeek, Groq, Mistral, etc.) under an OpenAI-compatible interface with virtual key management, budget enforcement, data-loss prevention (DLP), semantic caching, prompt governance, and Model Context Protocol (MCP) tool routing.
- 2. Endpoint AI Guard & Browser DLP (Client Plane):** Deploys a zero-bypass endpoint daemon (Raksha Browser Guard) on managed employee workstations (Windows and macOS). The guard transparently intercepts web-based interactions with consumer and enterprise generative AI websites (ChatGPT, Claude.ai, Gemini, Perplexity, DeepSeek, Copilot, etc.), enforcing real-time prompt redaction, sensitive file upload blocking, regulatory compliance policies, and automated audit logging.

[illegible]

2. Core Architectural Pillars

2.1 Dual-Plane Design

The server runs both planes inside a single high-performance Go executable compiled with Go 1.26+ and **valyala/fasthttp**:

- **Data Plane** (**/v1/***, **/openai/***, **/anthropic/***, **/genai/***): Handles ultra-low latency AI request proxying, streaming token aggregation, prompt transformations, caching, and rate limiting. Designed for sub-millisecond gateway overhead.
- **Control Plane** (**/api/***, **/ws**, **/health**): Provides the administrative backend, user identity lifecycle, role-based access control (RBAC), telemetry dashboards, MCP tool registry, and Browser AI agent control plane.

2.2 Endpoint Protection (Browser Guard)

- **Local Proxy Architecture**: Runs a local Python 3.11+ mitmproxy-based engine bound to loopback (**127.0.0.1:18103**) paired with a lightweight PAC (Proxy Auto-Configuration) daemon (**127.0.0.1:18195**).
- **Targeted Interception**: Network traffic to non-AI destinations is bypassed completely with zero latency impact. Only configured target websites (ChatGPT, Claude, etc.) route through the local inspection hook.
- **Fail-Closed / Fail-Open Modes**: Configurable behavior in enterprise policies. In strict compliance environments (**RAKSHA_FAIL_OPEN=0**), traffic to monitored targets is blocked if the guard daemon is halted.
- **DLP Engine**: Evaluates regex pattern dictionaries, keyword rules, and an optional local Ollama AI Guard Bot for semantic evaluation of prompt content before requests leave the employee workstation.

3. Technology Stack Reference

Subsystem	Technology / Library	Version / Baseline	Justification / Role
Server Runtime	Go (Golang)	1.26.4+	Memory efficiency, concurrent goroutine scheduling, zero-cost garbage collection tuning.
HTTP Engine	valyala/fasthttp	v1.58+	Zero-allocation HTTP server engine capable of handling 100k+ req/sec.
Routing	fasthttp/router	v1.5+	High-performance radix-tree HTTP request routing.
ORM / Data Access	GORM	v1.25+	Type-safe database abstraction across PostgreSQL, MySQL, and SQLite.
Primary Database	PostgreSQL	14.x / 15.x / 16.x	ACID compliance, JSONB support, partitioned log storage, and robust concurrency.
Client UI	React, TypeScript, Vite	React 19, TS 5.9, Vite 8	Enterprise single-page administrative dashboard.
State & UI Kit	Redux Toolkit, Radix UI, Tailwind	Tailwind CSS 4, Radix Primitives	Accessible UI components, responsive layout, dark/light theme management.
Guard Runtime	Python (Bundled Standalone)	3.11+	Mitmproxy addon framework, PyInstaller self-contained binary packaging.

Subsystem	Technology / Library	Version / Baseline	Justification / Role
Windows Packaging	Inno Setup	6.x	Automated Windows installer packaging with silent install flags and auto-update support.
macOS Packaging	Apple PackageMaker / pkgbuild	macOS Sonoma/Sequoia compatible	LaunchAgent deployment and system keychain root CA integration.
Containerization	Docker & Docker Compose	Docker 24+, Compose v2	Multi-stage image build and isolated multi-service orchestration.

4. Repository Structure & Module Breakdown

The codebase is organized into distinct Go modules, frontend applications, and endpoint agents:

```
unifai_project/ ■■■■ core/ # Core AI provider engine ■ ■■■■ providers/ # ~28 provider implementations ■ ■ ■■■■ openai/ # OpenAI Chat, Audio, Images, Embeddings ■ ■ ■■■■ anthropic/ # Anthropic Claude Messages API ■ ■ ■■■■ google/ # Google Gemini Pro / Flash SDK ■ ■ ■■■■ bedrock/ # AWS Bedrock Converse and InvokeModel ■ ■ ■■■■ ollama/ # Local Ollama client implementation ■ ■ ■■■■ azure/ # Azure OpenAI Service integration ■ ■■■■ schemas/ # Universal request/response schemas ■ ■■■■ mcp/ # Model Context Protocol client & code mode ■ ■■■■ keyselectors/ # Round-robin, weighted, and lowest-latency selection ■ ■■■■ network/ # HTTP client pool, retry policies, backoff ■ ■■■■ framework/ # Shared infrastructure modules ■ ■■■■ configstore/ # GORM database models & repository queries ■ ■ ■■■■ tables/ # GORM schema definitions for 40+ tables ■ ■■■■ logstore/ # LLM and Browser Guard request/response logs ■ ■■■■ rbac/ # 33 Resources × 6 Operations access engine ■ ■■■■ encrypt/ # AES-256-GCM symmetric database encryption ■ ■■■■ cluster/ # Multi-node cache and state replication ■ ■■■■ loadbalancer/ # Gateway load distribution algorithms ■ ■■■■ circuitbreaker/ # Upstream provider fault tolerance ■ ■■■■ featureflags/ # Dynamic runtime feature gating ■ ■■■■ plugins/ # Interceptor plugin implementations ■ ■■■■ guardrails/ # Prompt and response safety validation ■ ■■■■ semanticcache/ # Vector/hash-based response caching ■ ■■■■ telemetry/ # OpenTelemetry, Prometheus metrics exporter ■ ■■■■ governance/ # Token budget enforcement and rate limits ■ ■■■■ maxim/ # Prompt evaluation and observability ■ ■■■■ transports/ ■ ■■■■ raksha-http/ # Server endpoint and HTTP controllers ■ ■■■■ main.go # Binary startup, flags, environment loading ■ ■■■■ server/ # Fasthttp server initialization & middleware ■ ■■■■ handlers/ # 50+ API controller implementations ■ ■ ■■■■ auth.go # Login, session cookies, password resets ■ ■ ■■■■ browser_ai.go # Browser Guard telemetry and policies ■ ■ ■■■■ browser_ai_download.go # Endpoint installer packaging and distribution ■ ■ ■■■■ virtual_keys.go # Virtual key creation, budgets, and tokens ■ ■ ■■■■ models.go # Model catalog and provider management ■ ■ ■■■■ rbac.go # Roles, permissions, and access profiles ■ ■■■■ integrations/ # Pass-through provider route handlers ■ ■■■■ apps/browser-guard/ # Desktop Browser Guard client ■ ■■■■ agent/ # System tray, PAC daemon, and updater ■ ■ ■■■■ guard_bootstrap.py # Process lifecycle and cert verification ■ ■ ■■■■ guard_pac_server.py # Dynamic PAC file generator and server ■ ■ ■■■■ guard_tray.py # Cross-platform system tray status menu ■ ■■■■ proxy/ # Mitmproxy interception addon ■ ■ ■■■■ browser_ai_proxy.py # Traffic inspection, DLP rules, and redaction ■ ■ ■■■■ cert_installer.py # Automated Root CA installation in OS stores ■ ■■■■ installer/ # Windows Inno Setup script & macOS build scripts ■ ■■■■ release/ # Pre-built binaries, updaters, and VERSION.txt ■ ■■■■ ui/ # Single Page Application (React 19 / Vite) ■ ■■■■ app/ # Route structure & workspace views ■ ■ ■■■■ workspace/ # Authenticated admin & user dashboard ■ ■■■■ lib/ # Redux slices, RTK Query API clients ■ ■■■■ public/ # Logos, static assets, and favicon ■ ■■■■ configs/ # Configuration templates and model parameters ■■■■ deploy/ # Dockerfiles, docker-compose, and Helm charts ■■■■ docs/ # Project documentation & PDF artifacts
```

5. Security & Authentication Architecture

5.1 Dual-Token Authentication

- 1. **Dashboard Sessions (Browser):** Authenticated using HTTP-Only, Secure, SameSite cookies carrying encrypted session tokens. Protected against cross-site scripting (XSS) and cross-site request forgery (CSRF).

2. **API Access (Developer / CI/CD):** Authenticated via HTTP Authorization header: **Authorization: Bearer <VIRTUAL_KEY>**. Virtual keys are cryptographically generated 48-character strings tied to specific workspaces, rate limits, and token budgets.

5.2 Role-Based Access Control (RBAC) Matrix

The platform implements a multi-tier permission matrix combining:

- **System Roles:**
 - **admin:** Global administrator. Full access across all 33 resources and system settings.
 - **sub_admin:** Department / workspace administrator. Manages keys, policies, prompts, and Browser AI, but cannot alter master provider credentials or core infrastructure settings.
 - **user:** Standard employee. Read-only access restricted strictly to assigned sections (e.g., Prompt Repository, Personal Logs).
- **Custom Roles:** Granular control across 33 distinct system resources (**VirtualKeys**, **ModelProvider**, **GuardrailRules**, **Logs**, **Users**, etc.) mapped against 6 discrete operations (**Create**, **Read**, **Update**, **Delete**, **View**, **Download**).
- **Data Access Control (DAC):** Scopes records to **all-data** (organization-wide) or **own-data** (records created by or assigned to the authenticated user).

5.3 Cryptography & Secrets at Rest

All sensitive secrets—including third-party provider API keys, SMTP passwords, and external database credentials—are encrypted before storage in PostgreSQL using AES-256-GCM. The encryption key is derived from

UNIFAI_ENCRYPTION_KEY in **.env**.

5.4 Endpoint Guard Fleet Authentication

To prevent rogue network agents from polluting security telemetry or downloading enterprise DLP policies, the Browser Guard fleet uses a shared fleet secret (**RAKSHA_GUARD_SECRET**).

- Agents must send **X-Raksha-Guard-Secret: <SECRET>** on every heartbeat, policy fetch, and log upload.
- When **RAKSHA_GUARD_REQUIRE_SECRET=1**, the gateway enforces fail-closed rejection (HTTP 401/403) for any request lacking the valid secret.

6. Browser Guard Interception & Data Loss Prevention Engine

6.1 PAC (Proxy Auto-Configuration) Mechanism

The endpoint agent runs a lightweight HTTP server on **127.0.0.1:18195** that serves a dynamic PAC script to the local operating system:

```
function FindProxyForURL(url, host) { // Only route designated Generative AI domains to the local inspection proxy
  if (dnsDomainIs(host, "chatgpt.com") || dnsDomainIs(host, "claude.ai") || dnsDomainIs(host, "gemini.google.com") || dnsDomainIs(host, "deepseek.com") || dnsDomainIs(host, "perplexity.ai")) {
    return "PROXY 127.0.0.1:18103; DIRECT";
  } // All other enterprise and personal web traffic bypasses the proxy
  completely return "DIRECT";
}
```

6.2 Inspection & Rule Evaluation Pipeline

When an intercepted request hits the local proxy (**browser_ai_proxy.py**):

1. **Target Identification:** Matches hostname and URL path against downloaded enterprise **target_websites**.

2. **Body Extraction:** Extracts user prompt text and uploaded file attachments from multipart form-data or JSON payloads.
 3. **Rule Matching:** Evaluates active **guard_rules** in priority order:
 - **Regex Patterns:** Credit card numbers (Luhn algorithm), Social Security Numbers (SSN), API keys/tokens, AWS secrets, internal IP addresses.
 - **Keyword Dictionaries:** Confidential project codenames, source code markers, restricted customer names.
 - **AI Guard Bot:** Optional asynchronous semantic evaluation via local Ollama LLM.
 4. **Action Execution:**
 - **BLOCK:** Terminates request immediately, returning an HTTP 403 response with an enterprise branded violation message.
 - **REDACT:** Replaces sensitive matches with masking tokens (e.g., **[REDACTED_CREDIT_CARD]**) before forwarding upstream.
 - **WARN:** Allows request but logs a high-severity compliance alert to the management console and triggers email alerts.
 - **ALLOW:** Passes request unmodified and records standard telemetry.
-

7. Performance & Scalability Characteristics

- **Throughput:** Capable of proxying 25,000+ requests per second per server instance with under 1.2ms added latency overhead.
 - **Streaming Latency:** Server-Sent Events (SSE) chunks from upstream AI providers are forwarded directly to clients using zero-buffering fasthttp chunked transfers. Time-to-first-token (TTFT) degradation is below 2 milliseconds.
 - **Resource Footprint:**
 - Server Binary: ~45MB compiled executable. Idle RAM: ~65MB. Under full load: ~250MB - 1GB depending on cache size.
 - Endpoint Guard: Standalone Python daemon. Idle RAM: ~40MB. CPU utilization during active browsing: < 0.5%.
-

End of Technical Architecture Specification Document.